

Лабораторная работа №27. Знакомство с ASP.NET Core:

первый веб-сервер на C#

Цель работы: Установить .NET SDK и настроить проект в VS Code. Понять, что такое HTTP-запрос и HTTP-ответ. Создать минимальный веб-сервер на ASP.NET Core. Разобраться с маршрутизацией и middleware.

Краткая теория (вводная часть)

Представьте, что ваша WPF-программа не показывает окно, а ждёт, пока кто-то напишет ей через интернет. Когда браузер заходит на адрес `http://localhost:5000/`, он как раз «пишет» вашей программе. Она отвечает — и браузер показывает ответ. Это и есть веб-сервер.

Что такое ASP.NET Core?

ASP.NET Core — это современный, кроссплатформенный фреймворк от Microsoft для создания веб-приложений и API. Он работает поверх платформы .NET и заменяет устаревший ASP.NET Framework.

Основные особенности ASP.NET Core:

- работает на Windows, macOS, Linux;
- высокая производительность (входит в топ быстрых веб-фреймворков);
- встроенная поддержка DI (Dependency Injection);
- единая платформа для Web API, MVC, Razor Pages;
- легко разворачивается в Docker и облаке.

Сравнение backend-фреймворков

Фреймворк	Язык	Типизация	Компиляция	Скорость разработки	Производительность
Actix	Rust	Статич.	Да	Низкая	Максимальная
ASP.NET Core	C#	Статич.	Да	Высокая	Очень высокая
Fiber	Go	Статич.	Да	Очень высокая	Очень высокая
Fastify	JS/TS	Дин.	Нет	Максимальная	Высокая
Spring Boot	Java	Статич.	Да	Средняя	Высокая
Django	Python	Дин.	Нет	Максимальная	Низкая

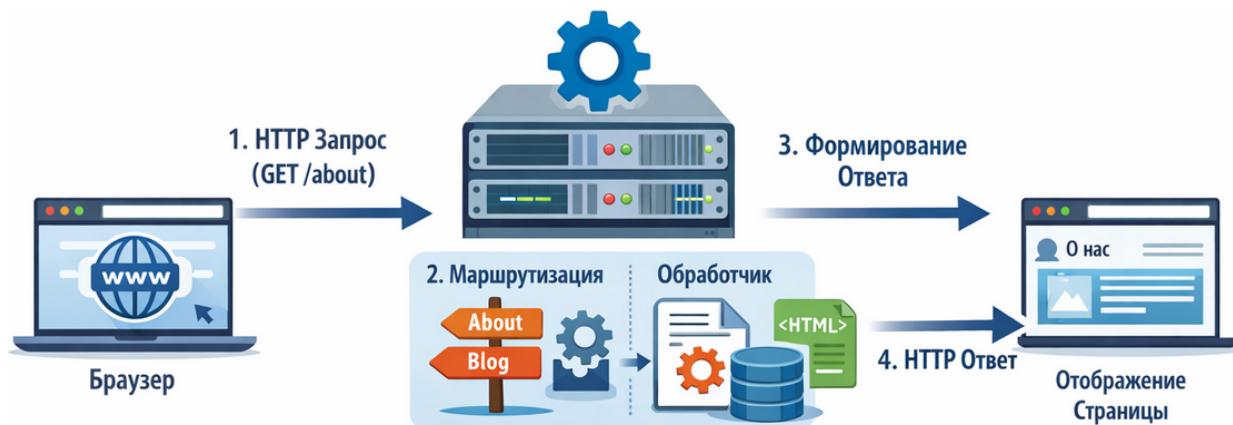
Как работает веб-сервер?

Когда вы открываете сайт в браузере, происходит следующее:

1. Браузер отправляет HTTP-запрос на сервер (например, `GET /about`).
2. Сервер получает запрос и определяет, какой обработчик вызвать (маршрутизация).

3. Обработчик выполняет нужную логику и возвращает HTTP-ответ.

4. Браузер получает ответ и отображает его пользователю.



HTTP-метод	Что делает
GET	Получить данные (загрузить страницу, список задач)
POST	Создать новый ресурс (отправить форму, добавить запись)
PUT	Обновить существующий ресурс целиком
DELETE	Удалить ресурс

Что такое middleware?

Middleware — это цепочка функций, через которую проходит каждый HTTP-запрос, прежде чем дойдёт до обработчика. Это аналог цепочки фильтров.

Сравнение с C#: представьте **middleware** как цепочку вызовов методов до **Main()**. Каждый middleware может обработать запрос, изменить его или передать следующему.

Шаг 1. Установка .NET SDK

1.1. Проверка наличия .NET

1. Откройте терминал в VS Code и выполните команду:

```
> dotnet --version
10.0.103
```

Если .NET уже установлен, вы увидите номер версии. Переходите сразу к Шагу 2.

Если команда не найдена — выполните установку.

1.2. Установка .NET SDK

1. Перейдите на официальный сайт: <https://dotnet.microsoft.com/download>

2. Скачайте .NET 10 SDK.

3. Установите, следуя инструкциям установщика.

4. После установки закройте и снова откройте терминал в VS Code.

5. Проверьте версию:

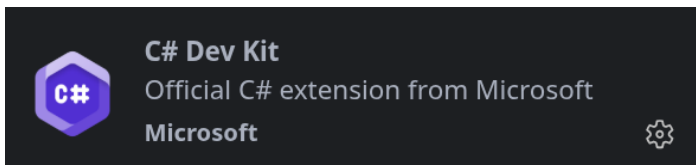
dotnet --version

dotnet --info

⚠ Убедитесь, что установлен именно SDK (Software Development Kit), а не только Runtime. SDK содержит всё необходимое для разработки, Runtime только для запуска.

1.3. Установка расширения C# в VS Code

1. Откройте **Extensions** (Ctrl+Shift+X).
2. Найдите **C# Dev Kit** (от Microsoft).



3. Установите расширение.

При необходимости VS Code попросит перезагрузку — перезагрузите.

Шаг 2. Подготовка рабочего окружения

1. Откройте любой терминал (например **Git Bash**).
2. Перейдите в каталог с документами.
3. Создайте директорию для лабораторной работы:

mkdir Lab27_AspNetCore

cd Lab27_AspNetCore

4. Создайте папку для скриншотов:

mkdir img

5. Откройте созданную папку в VS Code:

code .

Шаг 3. Инициализация Git-репозитория

1. Откройте терминал в VS Code (**Ctrl+`**)
2. Инициализируйте Git-репозиторий:

git init -b main

3. Создайте файл **.gitignore** для .NET проектов (чтобы не загружать лишние папки):

dotnet new gitignore

4. Создайте пустой файл README.md:

touch README.md

5. В VS Code для настройки переноса скобок на новые строки в C# нужно создать файл **.editorconfig** в корне с файлами проекта:

touch .editorconfig

6. Добавьте в ваш **.editorconfig** файл следующие правила (все скобки на той же строке):

```
[*.cs]
csharp_new_line_before_open_brace = none
```

7. Сделайте первый коммит:

git add .

git commit -m "Initial commit"

8. Создайте пустой public-репозиторий на **GitHub** с именем: **Lab27_AspNetCore**

9. Не добавляйте **README**, **.gitignore**, **License**.

10. Привяжите локальный репозиторий к удалённому и отправьте файлы:

git remote add origin <URL_репозитория>

git push -u origin main

11. Сделайте скриншот терминала с выполненными командами (commit, remote add, push) и сохраните в папку **img** с именем: **gitPushLab27_FIO.png**

Шаг 4. Создание проекта ASP.NET Core

4.1. Шаблоны .NET проектов

dotnet new создаёт проекты по шаблонам. Посмотрите доступные шаблоны:

dotnet new list

Нас интересуют два шаблона **web** и **webapi**:

Шаблон	Описание
web	Минимальный веб-сервер (Minimal API) — то, с чего мы начнём
webapi	Полноценный Web API с контроллерами — будем использовать позже
mvc	MVC-приложение с представлениями
razor	Razor Pages — сервер отдаёт HTML

4.2. Создание минимального веб-приложения

Создайте проект в текущей директории:

dotnet new web -n WebServer --no-restore

cd WebServer

Флаг *--no-restore* означает «не выполнять восстановление пакетов».

Изучите созданную структуру файлов:

```
>$ tree WebServer/
WebServer/
├── appsettings.Development.json
├── appsettings.json
├── Program.cs
├── Properties
│   └── launchSettings.json
└── WebServer.csproj
```

Назначение файлов:

- **Program.cs** - Точка входа приложения

- Здесь создаётся и запускается веб-сервер
- Настраиваются маршруты
- Добавляются middleware

Проще говоря: **главный файл, где собирается приложение**

- **WebServer.csproj** - Файл проекта

Содержит:

- версию .NET
- зависимости
- настройки сборки

Проще говоря: **описание проекта для компилятора**

- **appsettings.json** - Основной конфигурационный файл

Хранит:

- строки подключения
- настройки логирования
- параметры приложения

Проще говоря: **настройки приложения**

- **appsettings.Development.json** - Конфигурация для режима разработки

- переопределяет appsettings.json
- используется только при запуске в Development

Проще говоря: **настройки только для разработки**

- **Properties/launchSettings.json** - Настройки запуска

Содержит:

- порт
- URL
- переменные окружения
- режим запуска

Используется:

- Visual Studio
- Rider
- dotnet run

Проще говоря: **как запускать приложение**

4.3. Изучение файла Program.cs

Откройте файл **Program.cs** — вы увидите следующее:

```
// 1. Создание билдера
var builder = WebApplication.CreateBuilder(args);

// 2. Сборка приложения
var app = builder.Build();

// 3. Регистрация маршрута
app.MapGet("/", () => "Hello World!");

// 4. Запуск
app.Run();
```

Разберём каждую строку:

1. Это создание билдера (конструктора) приложения

Здесь:

- настраивается хост (Kestrel)
- подключается конфигурация (appsettings)
- настраивается DI-контейнер
- подключается логирование

То есть: создаётся среда для сборки приложения

2. Это сборка готового веб-приложения

На этом этапе:

- формируется middleware pipeline (Пайплайн — это регламентированный и документированный процесс выполнения типовых задач)
- создаётся HTTP-пайплайн обработки запросов

Из "чертежа" получается готовый сервер

3. Регистрируется маршрут

- HTTP метод: GET
- путь: /
- обработчик: лямбда-функция

Фактически: При GET / вернуть строку "Hello World!"

4. Запуск приложения

- стартует веб-сервер (Kestrel)
- начинается прослушивание HTTP-запросов

Приложение начинает принимать запросы

Итоговая таблица:

Код	Что делает
WebApplication.Create Builder(args)	Создание конфигулятора приложения. Подготовили сервер
builder.Build()	Сборка HTTP-пайплайна. Собрали сервер
app.MapGet("/", ...)	Добавление маршрута. Сказали как отвечать
app.Run()	Запуск сервера

Шаг 5. Первый запуск сервера

5.1. Восстановление зависимостей

1. Установите зависимости

dotnet restore

Если у вас возникнет ошибка (**данная ошибка возникает только на Linux**):

```

$ dotnet restore
/usr/share/dotnet/sdk/10.0.103/Sdks/Microsoft.NET.Sdk/targets/Microsoft.NET.Sdk.FrameworkReferenceResolution.targets(66,5): error NETSDK1226: данные пакета Prune не найдены .NETCoreApp 10.0 Microsoft.AspNetCore.App.
Чтобы игнорировать эту ошибку, установите для AllowMissingPrunePackageData значение true.

Восстановление сбой с ошибками (1) через 0,4 с

```

Это известная проблема ранних версий .NET 10, связанная с механизмом **PrunePackageReference** — оптимизацией, которая удаляет ненужные транзитивные зависимости. При dotnet restore SDK пытается найти данные для pruning пакетов у фреймворка Microsoft.AspNetCore.App. Но в некоторых версиях SDK эти данные отсутствуют или неправильно поставляются.

Быстрое решение

Вставьте эту строку внутрь блока **<PropertyGroup>**, как показано ниже в .csproj:

<AllowMissingPrunePackageData>true</AllowMissingPrunePackageData>

```

<PropertyGroup>
  <TargetFramework>net10.0</TargetFramework>
  <Nullable>enable</Nullable>
  <ImplicitUsings>enable</ImplicitUsings>
  +<AllowMissingPrunePackageData>true</AllowMissingPrunePackageData>
</PropertyGroup>

```

И снова:

dotnet restore

2. И запустите сервер (обратите внимание, что нужно запускать из папки WebServer):

dotnet run

3. В терминале вы увидите примерно следующее:

```

L>$ dotnet run
Используются параметры запуска из /home/denis/Project/ISRPO_Fullstack/Lab27_AspNetCore/W
launchSettings.json...
Сборка...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5050
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: /home/denis/Project/ISRPO_Fullstack/Lab27_AspNetCore/WebServer

```

Обратите внимание на порт. У вас может скорее всего будет другой, для примеров дальше я буду писать порт 5000, но вам нужно будем указывать свой!

Вы также можете вручную поменять порт в Properties/launchSettings.json (удобно для всех последующих лабораторных работ):

```

launchSettings.json
WebServer > Properties > launchSettings.json > {} profiles > {} http > applicationUrl
1 {
2   "$schema": "https://json.schemastore.org/launchsettings.json",
3   "profiles": {
4     "http": {
5       "commandName": "Project",
6       "dotnetRunMessages": true,
7       "launchBrowser": true,
8       "applicationUrl": "http://localhost:5000",
9       "environmentVariables": {
10        "ASPNETCORE_ENVIRONMENT": "Development"
11      }
12    },
13    "https": {
14      "commandName": "Project",
15      "dotnetRunMessages": true,
16      "launchBrowser": true,
17      "applicationUrl": "https://localhost:7050;http://localhost:5000",
18      "environmentVariables": {
19        "ASPNETCORE_ENVIRONMENT": "Development"
20      }
21    }
22  }
23 }

```

4. Откройте браузер и перейдите по адресу: **http://localhost:5000**

5. Вы должны увидеть текст: Hello World!

http://localhost:5050

Hello World!

6. Вернитесь в терминал — нажмите **Ctrl+C** чтобы остановить сервер.

5.2. Режим горячей перезагрузки

Вместо **dotnet run** используйте **dotnet watch** — сервер будет автоматически перезапускаться при изменении файлов: **dotnet watch run**

Практическое задание:

1. Запустите сервер командой:

dotnet watch run

2. Измените текст ответа на маршруте / на: **"Привет от ИСП-234! Автор: <ваше имя>"**

3. **Сохраните** файл и **обновите** страницу в браузере (**F5**)

4. Убедитесь, что браузер показывает новый текст

5. Сделайте скриншот браузера с новым ответом. Сохраните скриншот в папку **img** с именем: **step5_firstRunLab27_FIO.png**

Выполните в терминале (нажмите **Ctrl+C** чтобы остановить сервер):

cd ..

git add .

git commit -m "Step 5: first ASP.NET Core server running"

git push

cd WebServer

Шаг 6. Маршрутизация — несколько маршрутов

6.1. Добавляем маршруты

Откройте **Program.cs** и обновите маршруты:

```
// Маршрут 1: Главная страница
app.MapGet("/", () => "Добро пожаловать на сервер!");

// Маршрут 2: Информация о нас
app.MapGet("/about", () => "Это мой первый ASP.NET Core сервер");

// Маршрут 3: Текущее время сервера
app.MapGet("/time", () => $"Время на сервере: {DateTime.Now}");

// Маршрут 4: Маршрут с параметром в пути
app.MapGet("/hello/{name}", (string name) => $"Привет, {name}!");
```

Запустите сервер (**dotnet watch run**) и проверьте все маршруты в браузере:

URL	Ожидаемый результат
http://localhost:5000/	Главная страница
http://localhost:5000/about	Информация о сервере
http://localhost:5000/time	Текущее время
http://localhost:5000/hello/Иван	Приветствие с именем

6.2. Параметры маршрута

В маршруте `/hello/{name}` фигурные скобки означают параметр. ASP.NET Core автоматически извлекает значение из URL и передаёт его в обработчик.

Практическое задание:

1. Добавьте маршрут `/sum/{a}/{b}` — он должен возвращать сумму двух чисел

Подсказка: используйте `(int a, int b)` как параметры обработчика

2. Проверьте: `http://localhost:5000/sum/10/25` должно вернуть **35** (возможно потребуется перезагрузить сервер)

3. Сделайте скриншот браузера с результатом и сохраните в папку `img` с именем: **step6_routingLab27_FIO.png**

Выполните в терминале:

```
cd ..
```

```
git add .
```

```
git commit -m "Step 6: multiple routes with parameters"
```

```
git push
```

```
cd WebServer
```

Шаг 7. Возврат JSON из маршрута

7.1. Теоретическая часть

Веб-серверы чаще всего возвращают не текст, а данные в формате JSON — именно так работают API, которые мы использовали в JavaScript (`fetch`, `async/await`).

В ASP.NET Core достаточно вернуть объект C# — фреймворк автоматически сериализует его в JSON.

7.2. Пример с анонимным объектом

Добавьте в **Program.cs** следующие маршруты:

```
// Маршрут 6: Возвращаем JSON-объект
app.MapGet("/student", () => new {
    Name = "Иван Иванов",
    Group = "ИСП-234",
    Year = 3,
    IsActive = true
});
```

```
// Маршрут 7: JSON-массив
app.MapGet("/subjects", () => new[] {
    "РПМ",
    "РМП",
    "ИСПО",
    "СП",
});
```

Запустите сервер. Откройте <http://localhost:5000/student> в браузере — вы увидите JSON (**Обратите внимание, в зависимости от браузера, у вас может отличаться внешний вид, в моём случае используется Firefox**):

JSON	Необработанные данные	Заголовки
Сохранить	Копировать	Свернуть все
Развернуть все	Поиск в JSON	
name:	"Иван Иванов"	
group:	"ИСП-234"	
year:	3	
isActive:	true	

Обратите внимание: ASP.NET Core автоматически преобразовал имена свойств C# из PascalCase (Name) в camelCase (name). Это стандартный формат JSON.

Откройте <http://localhost:5000/subjects> в браузере — вы увидите JSON:

JSON	Необработанные данные	Заголовки
Сохранить	Копировать	Свернуть все
Развернуть все	Поиск в JSON	
0:	"РПМ"	
1:	"РМП"	
2:	"ИСПО"	
3:	"СП"	

7.3. Использование классов C#

Для реальных приложений лучше использовать именованные классы, а не анонимные объекты. Добавьте в конец файла **Program.cs** (после **app.Run()**):

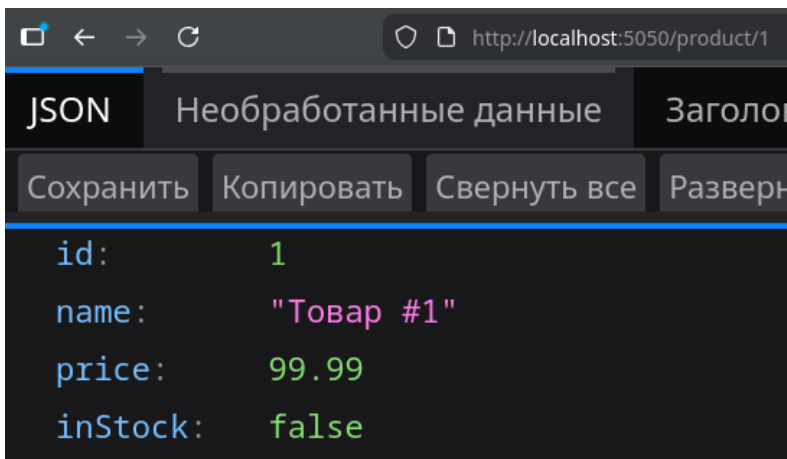
```
record Product(int Id, string Name, decimal Price, bool InStock);
```

record работает почти как *class*, но его главная цель — хранить данные, а не логику. Если знакомо: это как объект-конфигурация, у которого есть только поля и нет методов. Для возврата JSON из API это идеальный инструмент.

И новый маршрут перед `app.Run()`:

```
// Маршрут 8: JSON с использованием класса
app.MapGet("/product/{id}", (int id) => new Product(
    Id: id,
    Name: $"Товар #{id}",
    Price: id * 99.99m,
    InStock: id % 2 == 0
));
```

Запустите <http://localhost:5000/product/1> в браузере — вы увидите JSON:



Сделайте скриншот браузера с любым товаром и сохраните в папку **img** с именем:

step7_jsonLab27_FIO.png

Выполните в терминале:

cd ..

git add .

git commit -m "Step 7: returning JSON from routes"

git push

cd WebServer

Шаг 8. Middleware — цепочка обработки запросов

8.1. Теоретическая часть

Middleware — это функции, которые обрабатывают запрос до того, как он дойдёт до маршрута. Они выстраиваются в цепочку:

Запрос → [Middleware 1] → [Middleware 2] → [Маршрут] → Ответ

Каждый **middleware** может:

- выполнить действие до передачи запроса дальше;
- изменить запрос или ответ;

- прервать цепочку и вернуть ответ сразу (например, при ошибке авторизации);
- передать управление следующему middleware.

8.2. Встроенный middleware

Добавьте в **Program.cs** следующие **Middleware** (перед маршрутами):

```
// Middleware 1: Логирование запросов
app.Use(async (context, next) => {
    Console.WriteLine($"[LOG] {context.Request.Method} {context.
Request.Path}");
    await next(context); // передаём управление следующему
middleware
    Console.WriteLine($"[LOG] Ответ отправлен: {context.Response.
StatusCode}");
});
```

1 строка кода. Эта строка выполняется до того, как запрос пойдёт дальше по цепочке.

- Выводится информация о запросе:

- `context.Request.Method` — HTTP-метод
 - GET
 - POST
 - PUT
 - DELETE
- `context.Request.Path` — путь URL

2 строка кода. Передача запроса следующему middleware.

`next` — это делегат, который вызывает следующий middleware в конвейере.

Если не вызвать `next()`:

- запрос остановится
- дальше pipeline выполняться не будет

3 строка кода. Этот код выполняется после того, как маршрут вернул ответ.

`context.Response.StatusCode` — HTTP-статус ответа:

- 200 — OK
- 404 — Not Found
- 500 — Server Error
- 401 — Unauthorized

```
// Middleware 2: Добавляем заголовок в ответ
app.Use(async (context, next) => {
    context.Response.Headers.Append("X-Powered-By", "ASP.NET Core Lab27");
    await next(context);
});
```

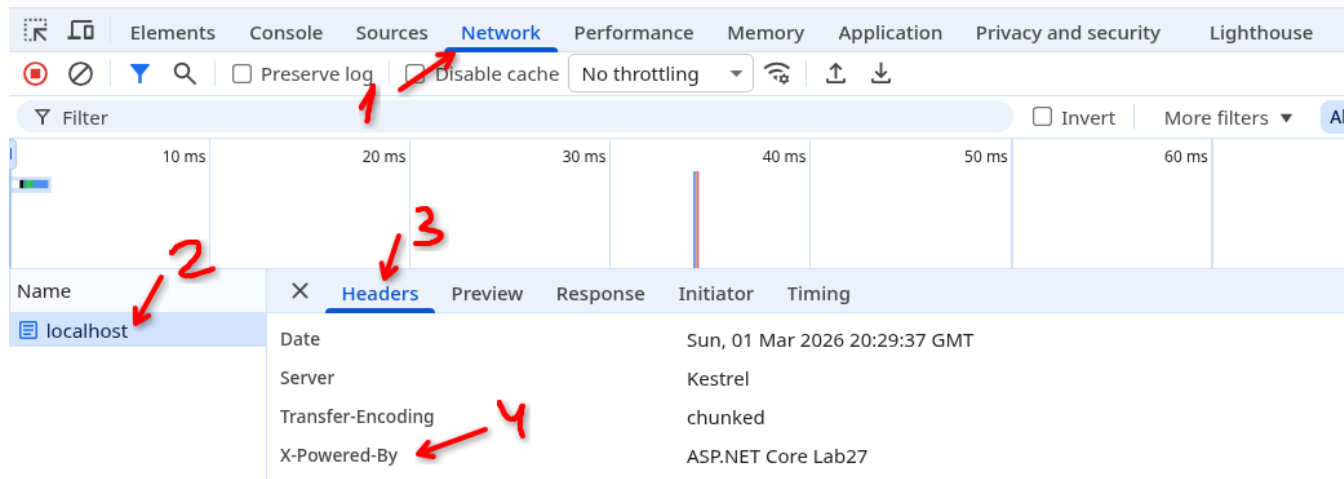
Строка добавления заголовка. Это означает, что сервер сообщает клиенту, на какой технологии работает приложение. X-Powered-By — это условное (пользовательское) имя HTTP-заголовка. Его часто используют как пример.

8.3. Проверка работы middleware

1. Запустите **dotnet watch run**
2. Откройте браузер и перейдите на <http://localhost:5000/>
3. Посмотрите на терминал — вы увидите логи нашего middleware:

```
[LOG] GET /
[LOG] Ответ отправлен: 200
```

4. Откройте DevTools браузера (F12) → **Network** → нажмите на запрос → **Headers**.
5. Убедитесь, что заголовок X-Powered-By: ASP.NET Core Lab27 присутствует.



⚠ *Порядок регистрации middleware важен! `app.Use()` добавляет middleware в цепочку в том порядке, в котором они объявлены. Никогда не регистрируйте маршруты (`MapGet` и т.п.) до middleware.*

Практическое задание:

1. Добавьте middleware, которое проверяет наличие параметра `?key=secret` в URL
2. Если параметр отсутствует или неверный — возвращайте статус 401:

Похоже, на этом сайте есть проблема

http://localhost:5050/ вернул ошибку.

Код ошибки: 401 Unauthorized

- Проверьте, правильно ли вы ввели адрес веб-сайта.

Попробовать снова

3. Если параметр верный — пропускайте запрос дальше

Подсказка:

```
Вызов middleware {  
    var key = context.Request.Query["key"];  
    (проверка != "secret") {  
        // TODO: установить статус 401 и прервать цепочку  
    }  
    await next(context);  
});
```

4. Проверьте: `http://localhost:5000/?key=secret` должен работать,

`http://localhost:5000/без ключа` — возвращать 401

Именно так работает простейшая защита API: без ключа сервер не отвечает. В реальных проектах вместо `?key=secret` используют заголовки авторизации, но принцип тот же.

5. Сделайте скриншот терминала с логами и скриншот DevTools с заголовком.

Сохраните в папку `img` с именами: **step8_middlewareLab27_FIO.png**

Выполните в терминале:

cd ..

git add .

git commit -m "Step 8: middleware logging and custom headers"

git push

cd WebServer

Шаг 9. Итоговый проект — мини-API сервера

9.1. Описание

Соберём всё изученное вместе. Создадим небольшой «сервер визитку» с несколькими маршрутами и **middleware**.

Выделите всё содержимое **Program.cs** между **builder.Build();** и **app.Run();** и закомментируйте - **Middleware**, **Маршруты**, **Product**, которые писали раньше, и перенесите их в конец файла **Program.cs**:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

//тут будем писать код

app.Run();

//record Product(int Id, string Name, decimal Price)
```

9.2. Финальный код Program.cs

Добавляем перед `app.Run()` Middleware - логирование:

```
app.Use(async (context, next) => {
    var method = context.Request.Method;
    var path = context.Request.Path;
    Console.WriteLine($"-> {method} {path}");
    await next(context);
});
```

Далее маршруты

1. Главная:

```
app.MapGet("/", () => Results.Ok(new {
    Message = "Добро пожаловать!",
    Version = "1.0",
    Time = DateTime.Now.ToString("HH:mm:ss")
}));
```

В ASP.NET Core `Results.Ok()` — это метод, который создаёт HTTP-ответ со статусом 200 (OK).

2. Информация о студенте:

```
app.MapGet("/me", () => Results.Ok(new {
    Name = "Иванов Иван",
    Group = "ИСП-234",
    Course = 3,
    Skills = new[] { "C#", "HTML", "CSS", "JS", "ASP.NET" }
}));
```

3. Калькулятор:


```
app.MapGet("/calc/{a}/{b}", (double a, double b) =>
Results.Ok(new {
    A = a,
    B = b,
    Sum = a + b,
    Diff = a - b,
    Mul = a * b,
    Div = b != 0 ? a / b : 0
}));
```

4. Маршрут не найден:

```
app.MapFallback(() => Results.NotFound(new {
    Error = "Маршрут не найден",
    Code = 404
}));
```

Проверьте работу всех маршрутов.

9.3. Проверка всех маршрутов

URL	Ожидаемый результат
GET /	Приветственное сообщение с временем
GET /me	Ваша карточка студента
GET /calc/15/3	Результаты всех операций
GET /несуществующий	Ответ 404 с JSON

Сделайте скриншоты всех маршрутов в браузере и сохраните в папку img с именами:

step9_finalLab27_FIO.png

Выполните в терминале:

cd ..

git add .

git commit -m "Step 9: complete mini-API server"

git push

Самостоятельное задание

Создать подробное и структурированное описание лабораторной работы №27 в файле README.md, используя Markdown.

Что должно быть в README.md:

Заголовок

Основная информация:

ФИО: Иванов Иван Иванович

Группа: ИСП-234

Дата: дд.мм.гггг

Краткое описание работы (что изучили)

Структура проекта

Список реализованных маршрутов с описанием

Главные выводы (минимум 3)

Итоговая таблица (обязательно включить в README.md):

Характеристика	ASP.NET Core
Создание сервера	WebApplication.CreateBuilder()
Запуск	app.Run()
Маршрут GET	app.MapGet("/", fn)
Параметр маршрута	{name} → (string name)
Возврат JSON	return new {...} / Results.Ok(...)
Middleware	app.Use(async (ctx, next) => ...)
Логирование	Встроенное + Console.WriteLine
Статус ответа	Results.NotFound(...)
Тип данных	Строгие (C# record, class)

Найдите в интернете реальное использование фреймворка ASP.NET Core на крупных проектах. Запишите ответ в README.md в раздел «Интересные факты».

Ответьте на вопросы:

1. Чем Minimal API отличается от MVC в ASP.NET Core?
2. Что произойдёт, если поставить `app.MapGet(...)` до `app.Use(...)`?
3. Почему ASP.NET Core преобразует PascalCase в camelCase при сериализации JSON? Это исследовательский вопрос — поищите ответ в документации или статьях
4. Что означает код ответа HTTP 401? А 404? А 200?
5. Чем **dotnet run** отличается от **dotnet watch run**?

После выполните в терминале:

git add .

git commit -m "Added complete README for Lab27"

git push

Итоговая таблица: что изучили в лабораторной

Команда / метод	Что делает
dotnet --version	Проверить версию .NET SDK
dotnet new web	Создать минимальное веб-приложение
dotnet run	Запустить сервер
dotnet watch run	Запустить с автоперезагрузкой при изменениях
dotnet restore	Восстановить пакеты/зависимости
dotnet new gitignore	Создать .gitignore для .NET-проекта
app.MapGet(path, fn)	Зарегистрировать GET-маршрут
app.Use(middleware)	Добавить middleware в цепочку
Results.Ok(data)	Вернуть ответ 200 с данными
Results.NotFound(data)	Вернуть ответ 404 с данными